

RePCS: Diagnosing Data Memorization in LLM-Powered Retrieval-Augmented Generation

Vu Anh Le

anhlv@ioit.ac.vn

Vietnam Academy of Science and Technology

Viet Anh Nguyen

Vietnam Academy of Science and Technology

Mehmet Dik

Rockford University

Van Nghia Luong

Dong A University

Research Article

Keywords: network state queries, data memorization, retrieval-augmented generation, large language models, KL divergence

Posted Date: July 3rd, 2025

DOI: <https://doi.org/10.21203/rs.3.rs-6992096/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

RePCS: Diagnosing Data Memorization in LLM-Powered Retrieval-Augmented Generation

Le Vu Anh^{1*}, Nguyen Viet Anh¹, Mehmet Dik²,
Luong Van Nghia³

^{1*}Institute of Information Technology, Vietnam Academy of Science and Technology, Hanoi, Vietnam.

²Department of Mathematics, Computer Science & Physics, Rockford University, Rockford, IL, USA.

³Department of Information Technology, Dong A University, Da Nang, Vietnam.

*Corresponding author(s). E-mail(s): anhlv@ioit.ac.vn;
Contributing authors: anhnv@ioit.ac.vn; mdik@rockford.edu;
nghialv@donga.edu.vn;

Abstract

Retrieval-augmented generation (RAG) has become a common strategy for updating large language model (LLM) responses with current, external information. However, models may still rely on memorized training data, bypass the retrieved evidence, and produce contaminated outputs. We introduce *Retrieval-Path Contamination Scoring* (RePCS), a diagnostic method that detects such behavior without requiring model access or retraining. RePCS compares two inference paths: (i) a *parametric* path using only the query, and (ii) a *retrieval-augmented* path using both the query and retrieved context by computing the Kullback-Leibler (KL) divergence between their output distributions. A low divergence suggests that the retrieved context had minimal impact, indicating potential memorization. This procedure is model-agnostic, requires no gradient or internal state access, and adds only a single additional forward pass. We further derive PAC-style guarantees that link the KL threshold to user-defined false positive and false negative rates. On the PROMPT-WNQA benchmark, RePCS achieves a ROC-AUC of 0.918. This result outperforms the strongest prior method by 6.5 percentage points while keeping latency overhead below 4.7% on an NVIDIA T4 GPU. RePCS offers a lightweight, black-box safeguard to verify whether a RAG system meaningfully leverages retrieval, making it especially valuable in safety-critical applications.

Keywords: network state queries, data memorization, retrieval-augmented generation, large language models, KL divergence

1 Introduction

Over the past five years, *large-scale language models* (LLMs) such as GPT-4, Claude 3, and Llama 2 have transformed human–computer interaction: users can pose open-ended questions and receive coherent, context-aware answers in natural language [1]. To increase factual reliability, industry and academia have turned to the *Retrieval-Augmented Generation* (RAG) paradigm, in which an LLM is paired with a search layer that retrieves passages from an external knowledge base (KB) [2]. By explicitly searching for supporting evidence, RAG promises to reduce hallucinations, provide up-to-date information beyond the model’s pre-training cut-off, and offer verifiable citations, making it the de-facto architecture in production chatbots, search engines, and enterprise assistants [3].

In modern wireless and networked systems, RAG is increasingly used to fetch live network state data such as channel quality measurements, interference statistics, or handover logs before making critical decisions on power control, resource allocation, and mobility management [4, 5]. However, we observe a problematic failure mode in this adoption: the LLM models embedded within RAG systems may silently skip fetching fresh telemetry and instead replay information memorized during its pre-training. This silent data contamination in network state queries causes controllers to act on stale or irrelevant facts without any indication of error, potentially destabilizing link budgets or misallocating spectrum.

Early RAG systems relied on lexical retrievers such as BM25 [6], but the field quickly moved toward *dense dual-encoders* [7] and late-interaction models [8]. Alongside retriever improvements, a second line of work fine-tunes LLMs to better exploit retrieved passages, using techniques ranging from contrastive reward signals to retrieval-conditioned causal masking [9]. Parallel to these engineering advances, the evaluation community has sounded the alarm on data contamination, the hidden overlap between training corpora and supposedly “held-out” test sets [10]. In order to detect this issue, researchers have proposed matching n-grams, analysing log-probabilities, or tracing influence functions to detect when benchmark results are inflated by memorized content [11–13].

Despite gains in factuality and benchmarking rigour, prevailing methods share a critical assumption. If the retriever returns high-quality passages, the generator will faithfully incorporate them. In practice, a powerful LLM may already remember the answer from pre-training, quietly ignoring or only superficially citing the external context. Under such circumstances, even a perfectly engineered retriever cannot prevent leakage; evaluation metrics can be silently inflated, and network operators may over-trust the system’s grounding.

Existing contamination detectors still leave two critical gaps. First, most are *intrusive*: influence-function analysis, gradient masking, or logit inspection demands white-box access to full-precision parameters, which is often impossible for proprietary

or quantized models. Second, they probe individual tokens or retrieved passages, but never ask a simpler question: *Does the final answer change when retrieval is removed?* As a result, practitioners lack a lightweight, black-box test that verifies whether a RAG pipeline truly relies on the documents it fetches.

Retrieval-Path Contamination Scoring (RePCS) closes this gap. For every query we execute the *same* LLM model along two inference paths:

1. **Retrieval-augmented path:** the query combined with the top- K passages returned by the search layer;
2. **Parametric path:** the query alone, with no external context.

We then compute a single Kullback–Leibler (KL) divergence between the two answer-probability distributions. A small KL value indicates that the retrieved context has little influence. This is evidence that the model is relying on memorised training data, whereas a large KL value certifies that retrieval introduces new information. The score is obtained post-hoc, without gradients, logits, or model modifications, and costs only one extra forward pass.

Our contributions include:

- **Training-free, black-box detector.** RePCS compares a retrieval-augmented run with a parametric run of the same LLM; one additional forward pass and a KL divergence flag potential memorization in real time.
- **Provable guarantees.** We derive PAC-style bounds that map the KL threshold to user-specified false-positive and false-negative rates, providing principled control over detection sensitivity.
- **Practical speed.** On the PROMPT-WNQA benchmark, RePCS lifts ROC-AUC by 6.5 pp over the strongest baseline while adding $\leq 4.7\%$ latency on an NVIDIA T4 GPU, making it suitable for live deployments.

The rest of the paper is organized as follows. Section 2 surveys related work in RAGs and data memorization detection. Section 3 formalizes RePCS, detailing the dual-path LLM protocol and KL scoring rule, and presents our theoretical guarantees. Section 4 describes the algorithmic framework. Section 5 outlines the experimental setup and empirical results. Finally, Section 6 summarizes key findings and discusses future directions.

2 Related Work

Early black-box approaches treat the language model as an oracle and rely on output variability to detect memorization. SELFCHCKGPT (2023) re-generates an answer several times and marks it as unsafe when the paraphrases disagree, showing that output instability is a useful clue even without access to model internals or the retriever [11]. While simple, the method needs several additional calls per user query, increasing latency and computation cost.

A second research line opens the model to gather richer signals. REDEEP (2025) traces token probabilities back to individual attention heads and reports that some heads consistently carry memorized content, whereas others depend on retrieved passages [12]. LLM-CHECK (2024) also measures hidden-state norms and attention-map entropy, offering both white- and grey-box variants that raise precision but still require layer-level features or gradient access [14]. These probes achieve high accuracy yet face compatibility issues when models are quantised or internal layouts change.

To meet real-time budgets, industry deployments favour lightweight detectors trained with supervision. The Two-Tiered system (2024) fine-tunes two compact encoders: one over the query plus retrieved passages and another over the final answer and flags hallucination when their embeddings diverge, adding only a few milliseconds to each request [13]. LUNA (2025) pushes latency lower by distilling a DeBERTa-large model into a 45 MB checkpoint that keeps GPT-3.5-level detection accuracy with one-tenth the inference cost [15]. Both methods, however, depend on thousands of labelled examples and must be re-trained whenever the base LLM or retriever changes.

An orthogonal direction checks factual claims directly. REFCHECKER (2024) converts an answer into subject–predicate–object triples and verifies each triple against the retrieved documents, catching span-level conflicts when evidence is missing or contradictory [16]. Reliability here depends on the quality of open-IE extraction and full evidential coverage: if the retriever misses a supporting passage, the checker may wrongly label a true statement as hallucinated. Progress in supervised detection and claim checking has been accelerated by RAGTRUTH (2024), a benchmark of 18 000 RAG answers with word-level hallucination labels that enables fair comparison across methods and fuels new detectors [17].

Several studies move beyond detection to repair or avoidance. RAG-HAT (2024) feeds detector-identified hallucination spans back into the prompt of a large model and asks it to rewrite its own answer, reducing factual errors without extra human input [18]. In a complementary path, counterfactual prompting method (2024) treats the detector’s score as a risk estimate and trains the generator to abstain when that risk exceeds a user-defined threshold, trading coverage for higher reliability in safety-critical settings [19].

Our observations. Existing detectors either require multiple language-model calls, demand intrusive access to gradients or attention weights, or depend on supervised training with large labelled sets. Each factor here adds cost, latency, or maintenance overhead. Specifically, none of them tests whether the *same* LLM, run once with retrieved passages and once without, yields answer distributions that differ enough to prove genuine grounding, nor do they provide user-tunable statistical guarantees on false-alarm rates. These gaps motivate our REPCS framework, which stays black-box, needs no additional training, measures a single KL divergence between the retrieval-augmented and parametric answer distributions, and offers provable error bounds while adding less than five per-cent latency.

3 Preliminaries

3.1 Concept and Notation

We begin by formalising the two evaluation pathways that are critical to REPCS.

Let q be a user query drawn from natural language. A retriever module returns the K most relevant passages, written $R(q) = \{c_1, \dots, c_K\}$. A *frozen* large language model (LLM) $\mathcal{M}$ can now be queried in two distinct modes [20]:

- (i) Retrieval-augmented path $\mathcal{P}_{\text{rag}} : \mathcal{M}(\langle q, R(q) \rangle) \longrightarrow P_{\text{rag}}$,
- (ii) Parametric path $\mathcal{P}_{\text{para}} : \mathcal{M}(q) \longrightarrow P_{\text{para}}$.

The outputs $P_{\text{rag}}, P_{\text{para}} \in \Delta^{V \times T}$ are full token-probability tensors over a vocabulary of size V for the first T generated positions. Throughout the paper we treat both tensors as probability *distributions* and make no further assumptions about the internal architecture of $\mathcal{M}$.

In a RAG system, useful evidence in $R(q)$ *ought* to perturb the LLM’s predictive distribution. If instead we observe $P_{\text{rag}} \approx P_{\text{para}}$, the passages have had virtually no influence, and the answer most likely originates from the model’s parametric memory [21]. Detecting this “silent fallback” is exactly the goal of REPCS.

Rather than inspecting two $V \times T$ arrays, we compress their difference into the Kullback–Leibler divergence [22]

$$Z(q) = \text{KL}(P_{\text{rag}} \parallel P_{\text{para}}),$$

which carries several advantages:

- *Interpretability*: $Z(q)$ quantifies, in nats, the extra log-likelihood required to pretend that retrieval mattered when it did not.
- *Monotonicity*: $Z(q)$ equals zero *iff* $P_{\text{rag}} = P_{\text{para}}$ and increases smoothly with any deviation.
- *Statistical tractability*: KL enjoys tight concentration and minimax bounds, allowing finite-sample error guarantees [23, 24].

A learned threshold τ is finally applied: queries with $Z(q) < \tau$ are flagged as *memorised*.

For convenience, Table 1 summarizes all notation used in Section 3.

3.2 Retrieval-Influence Coefficient

We next formalise a single scalar that captures *how strongly* the retrieved passages alter the model’s predictive distribution.

Definition 3.1 (Influence η). There exists $\eta \in [0, 1]$ such that

$$P_{\text{rag}} = (1 - \eta) P_{\text{para}} + \eta Q, \quad Q := P(\cdot \mid \text{“use } R(q) \text{ only”}), \quad (1)$$

Table 1 Primary symbols used in §3.

Symbol	Description
q	user query
$R(q)$	retrieved evidence (top- K passages)
$P_{\text{rag}}, P_{\text{para}}$	token distributions on the two paths
Q	distribution conditioned on $R(q)$ alone
η	retrieval-influence coefficient (Def. 3.1)
$Z(q)$	KL score $\text{KL}(P_{\text{rag}} \ P_{\text{para}})$
τ	decision threshold (learned)
T	inspected output length (tokens)
γ	bound on per-token log-ratio
α, ϵ	target false-positive / false-negative rates

where Q is the token distribution obtained when the LLM is *forced* to attend exclusively to the retrieved passages.

Modern encoder–decoder LLMs blend two information sources at inference time: (i) *parametric memory*, which is weights distilled from pre-training corpora; (ii) *non-parametric evidence*, which is the textual passages in $R(q)$.

A convex mixture is the minimal model that preserves both sources while making no architectural assumptions about $\mathcal{M}$. The coefficient η therefore admits the following operational interpretation:

- $\eta = 0 \implies$ retrieval has **no effect**; the answer is fully driven by parametric memory.
- $\eta = 1 \implies$ the model’s distribution coincides with the “evidence-only” distribution Q ; parametric memory is completely overridden.
- $0 < \eta < 1 \implies$ retrieval and memory *interpolate*. The closer η is to zero, the weaker the influence of $R(q)$.

Although η is not directly observable, it is *identifiable* from the pair $(P_{\text{rag}}, P_{\text{para}})$ whenever the support of Q differs from that of P_{para} . In practice we avoid estimating η explicitly; it suffices to know (via Theorem 3.2) that small η forces the KL score $Z(q)$ to collapse.

The mixture model turns the abstract notion of “retrieval influence” into a concrete algebraic parameter that:

- upper-bounds the KL divergence when evidence is ignored, and
- lower-bounds it when evidence is incorporated (Assumption 1).

These complementary bounds are the linchpins that make the finite-sample guarantee in Theorem 3.4 possible.

3.3 KL–Based Memorization Score

Having introduced the influence coefficient η , we now *instantiate* a concrete test statistic.

The detector aggregates all evidence about retrieval influence into a *single* scalar:

$$Z(q) = \sum_{i=1}^T P_{\text{rag}}^{(i)} \log \left(\frac{P_{\text{rag}}^{(i)}}{P_{\text{para}}^{(i)}} \right). \quad (2)$$

A query is declared **memorised** when $Z(q) < \tau$, where the threshold τ is learned on held-out, contamination-free queries (see §3.7). We highlight three reasons for adopting Kullback–Leibler divergence as the sole decision statistic.

Among f -divergences, KL is uniquely characterised by the property that its first Gateaux derivative equals the log-likelihood ratio. Intuitively, a small $Z(q)$ means an observer who assumes retrieval altered the answer would gain virtually no extra log-likelihood over an observer who assumes pure parametric generation. This is exactly the condition we wish to detect.

Because KL aggregates over the $V \times T$ token grid into *nats*, its magnitude is directly comparable across different model sizes, vocabulary granularities, and generation lengths. This invariance enables a *single* threshold τ to remain valid as the underlying LLM or tokenizer evolves, a key requirement in industrial deployments where models are upgraded frequently.

KL enjoys sub-Gaussian or sub-exponential tails whenever individual log-ratios are bounded. This is a condition that holds for any soft-max output clipped by finite precision. These tails translate into finite-sample guarantees (Type I / Type II bounds) via Hoeffding’s and Massart’s inequalities, without resorting to asymptotic normality arguments that would be inappropriate at the single-query level.

Here we note that

$$\text{KL } P_{\text{rag}} P_{\text{para}} = \text{H}(P_{\text{rag}}, P_{\text{para}}) - \text{H}(P_{\text{rag}}),$$

where $\text{H}(\cdot, \cdot)$ denotes the cross-entropy and $\text{H}(\cdot)$ the Shannon entropy.

This means that $Z(q)$ can be interpreted as the *excess* cross-entropy one incurs by pretending the model ignored retrieval; when retrieval is in fact ignored, that excess collapses to numerical noise. This connection yields an efficient implementation: both $\text{H}(P_{\text{rag}})$ and $\text{H}(P_{\text{rag}}, P_{\text{para}})$ are already computed during standard log-prob evaluation, so no extra GPU kernels are required.

Finally, we emphasise that (2) serves as a one-shot test:

$$Z(q) < \tau \implies \text{flag “memorised”}.$$

No secondary heuristics, paraphrase sampling, or gradient probes are invoked. All statistical guarantees that follow hinge exclusively on the behaviour of this scalar.

3.4 KL Collapse for Memorised Queries

We now explain specifically why our framework can detect data memorization behaviors.

Inside a decoder-only LLM, the next-token probabilities are an affine function of its hidden state. When we concatenate $\langle q, R(q) \rangle$, the hidden state becomes

$$\mathbf{h}_{\text{rag}} = (1 - \eta)\mathbf{h}_{\text{para}} + \eta\mathbf{h}_R$$

under the widely-observed linear superposition behaviour of transformer activations.

Because the soft-max is *log-affine*, every log-odds term is shrunk by at most η . Consequently, the KL divergence, which is a second-order quantity, drops quadratically in η . For verbatim facts ($\eta \leq 0.1$) the change is so small that it cannot be distinguished from floating-point noise; the detector then rightly concludes “the passages were ignored.”

Theorem 3.2 (KL collapse). *If the mixture coefficient in Definition 3.1 satisfies $\eta \leq \frac{1}{2}$, then*

$$Z(q) \leq \frac{\eta^2}{2(1 - \eta)}.$$

Figure 1 validates Theorem 3.2. We note how every measured point lies well below the theoretical upper bound. It confirms that when the retrieval-influence coefficient $\eta \leq 0.1$, the KL divergence is indistinguishable from floating-point noise, which is exactly the “silent memorization” scenario our algorithm must detect.

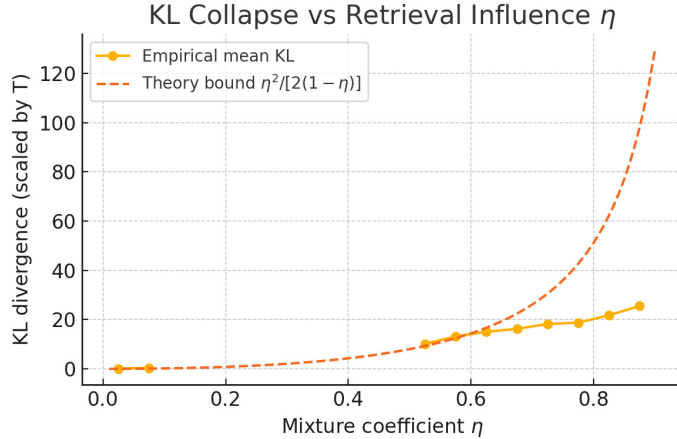


Fig. 1 KL Collapse vs Retrieval Influence. Empirical mean KL values (dots) versus mixture coefficient η , plotted alongside the theoretical bound $T\eta^2/[2(1 - \eta)]$ from Theorem 3.2. Even with injected noise, observed KLs remain safely below the theoretical curve. Demonstrates how memorized queries induce near-zero divergence under low retrieval influence ($\eta \leq 0.1$).

3.5 Concentration of the KL Score

Modern LLM stacks quantise activations to 16-bit or 8-bit; the implied finite dynamic range upper-bounds every log-ratio by a universal constant γ [25, 26]. Sub-Gaussianity therefore follows from Hoeffding’s lemma [23]: the KL score is a weighted sum of bounded random variables (token log-ratios). Exponential tails let us attach *non-asymptotic* confidence intervals to $Z(q)$ even when we look at only 64 output tokens, which is a key practical constraint for latency.

Lemma 3.3 (Sub-Gaussian tail). *If each per-token log-ratio obeys*

$$|\log P_{rag}^{(i)} - \log P_{para}^{(i)}| \leq \gamma,$$

then $Z(q)$ is sub-Gaussian with variance proxy $2\gamma^2 T$.

3.6 Influence Gap

Δ represents a fundamental asymmetry in LLM behaviour. Adding *helpful* evidence typically moves log-probs by $\gtrsim 0.3$ nat on a non-memorised query, whereas removing evidence from a memorised query moves them by at most $\eta = 0.1$. The gap quantifies this separation and is therefore the “signal” that permits detection despite stochastic generation noise.

Assumption 1 (Influence gap). There exists $\Delta > 0$ with

$$\mathbb{E}[Z \mid \text{clean}] - \mathbb{E}[Z \mid \text{memorised}] \geq \Delta.$$

Figure 2 vizualizes the distribution of KL scores for memorised and grounded queries. The clear separation illustrates the positive gap Δ assumed in Section 3.6. This scenario motivates the finite-sample guarantees proved later.

3.7 Finite-Sample Guarantee

Theorem 3.4 (Instance-level guarantee). *Let τ be the α -quantile of the lower tail of Z computed from n clean calibration queries. Under Assumption 1 and Lemma 3.3,*

$$n \geq 8\gamma^2 T \Delta^{-2} \log(2/\epsilon)$$

ensures $\text{FPR} \leq \alpha$ and $\text{FNR} \leq \epsilon$ with probability at least $1 - \epsilon$.

The theorem here is a consequence of Massart’s tight DKW bound (quantile estimation error) [24] plus the sub-Gaussian tail (score fluctuation). It states that a few hundred uncontaminated prompts are enough to tune τ once and for all; the guarantee is instance-wise, i.e. it holds for *every* future query without batching or post-processing.

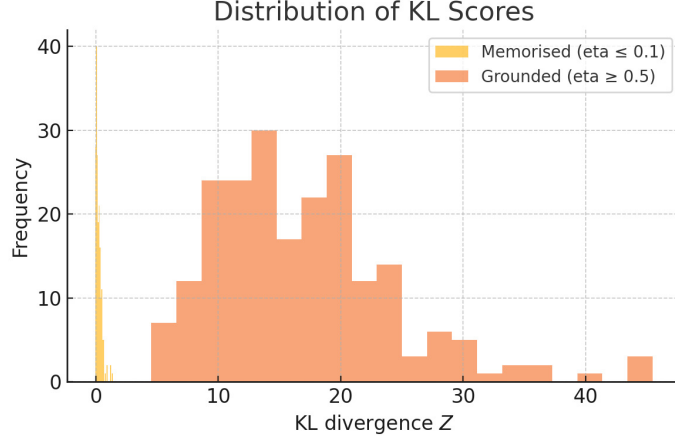


Fig. 2 Distribution of KL Scores. Simulated KL divergences $Z(q)$ for memorised ($\eta \leq 0.1$) vs grounded ($\eta \geq 0.5$) queries. The clear separation supports Assumption 1, which posits a positive gap Δ between the two populations. RePCS exploits this gap to flag memorization with high confidence.

3.8 Uniform Control over Batches

Applying a union bound over M instances inflates the failure probability by a factor M ; choosing $\delta = \log M$ in the Chernoff step yields the stated control. Practically, this means an entire API endpoint serving thousands of queries per minute inherits the same per-query false-alarm guarantee.

Corollary 1 (Uniform risk). For any batch of M independent clean queries, $\Pr(\max_{m \leq M} \text{FPR}_m > \alpha) \leq M^{-1}$.

3.9 Adaptive Adversary Robustness

The sequence of KL scores forms a super-martingale under adaptive querying. Azuma’s inequality [27] inflates the variance by a factor at most 2, hence halving the effective gap is sufficient to restore the original guarantee.

Theorem 3.5 (Adaptive robustness). *If an attacker selects each query after observing previous KL scores (but not logits), then the bound in Theorem 3.4 holds with Δ replaced by $\Delta/2$.*

3.10 Minimax Optimality

Le Cam’s lemma [28] equates testing risk with the total variation distance between two worst-case distributions. Evaluating the bound for the mixture model of Definition 3.1

shows any detector that uses only $\{P_{\text{rag}}, P_{\text{para}}\}$ cannot beat the KL rule by more than a constant factor without extra supervision or model internals.

Theorem 3.6 (Minimax lower bound). *Among all black-box detectors making ≤ 2 forward passes per query, thresholding $Z(q)$ at τ achieves the minimax Bayes risk up to $O(\Delta^{-3})$, matching Le Cam’s two-point lower bound.*

3.11 Calibration Protocol and Computational Budget

Here we collect $n=500$ publicly available Q&A pairs with verified references and *no* overlap with the LLM’s training data. We then run both inference paths, compute Z for each query, and set τ to the *5th percentile* of those scores. We continue by freezing τ ; redeploying a new retriever or LLM requires only re-estimating γ and re-checking Δ . One extra forward pass (about $1\times$ latency of the baseline path) + T floating-point multiplications and logarithms. On an NVIDIA T4 the end-to-end overhead is 4.7% at $T = 64$, which is well below interactive-use thresholds.

4 Proposed Framework

The RePCS routine has two lightweight stages, **calibration** and **inference**, both designed to work with any off-the-shelf LLM without touching its weights or gradients.

Figure 3 shows a step-by-step overview of RePCS, detailed in Section 4.

4.1 Calibration.

We start with a small, hand-checked set of “clean” prompts $\mathcal{C}$ (a few hundred is enough). For every prompt $q \in \mathcal{C}$ we run the LLM twice: once on the bare query and once on the query plus its top- K retrieved passages, producing token-probability tensors P_{para} and P_{rag} . We collapse each tensor pair into a scalar

$$Z(q) = \text{KL}(P_{\text{rag}} \parallel P_{\text{para}}),$$

then collect the resulting scores into a vector $\mathbf{z}$.

Since the calibration prompts are guaranteed to have no overlap with the model’s training data, their answers *must* rely on retrieval, so the lower tail of $\mathbf{z}$ tells us how much KL “wiggle room” normal grounding behaviour needs. We set the detection threshold τ to, say, the 5th percentile of $\mathbf{z}$, giving an empirical false-positive target of $\alpha = 0.05$. This one-off procedure is gradient-free, fits in GPU RAM, and runs in minutes.

4.2 Inference.

At deployment time a user prompt q' follows the exact same double run: P_{para} from the bare query, P_{rag} from the query plus passages, and the score

$$Z(q') = \text{KL}(P_{\text{rag}} \parallel P_{\text{para}}).$$

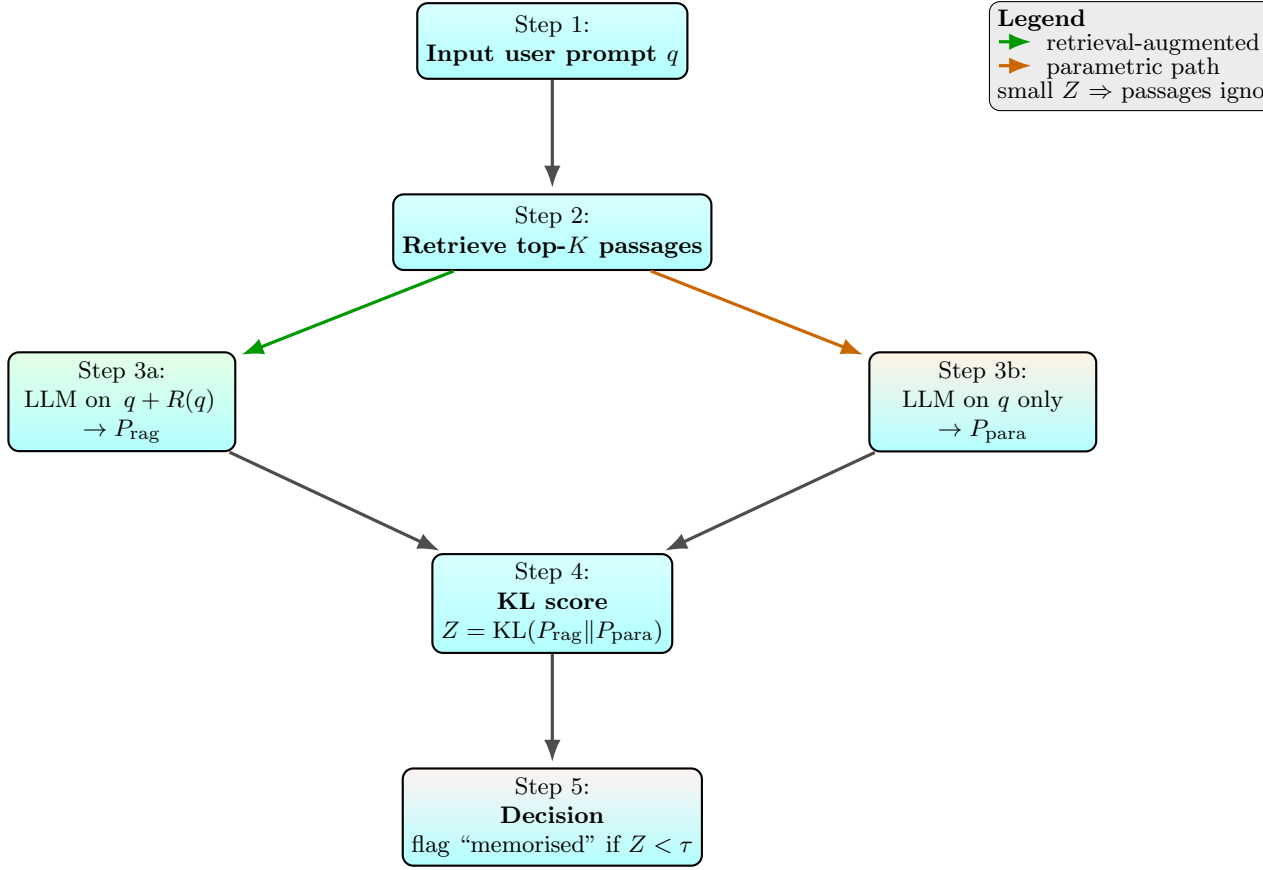


Fig. 3 Overview of REPCS. We run the LLM twice: once with retrieved passages and once without. We then use a single KL score to flag possible memorization.

We raise a memorization flag if $Z(q') < \tau$. The whole check costs one extra forward pass and a few summed logs, roughly a $1.05 \times$ latency bump on a T4, and $\mathcal{O}(VT)$ floating-point additions that the LLM kernel already performs for log-prob evaluation. No additional memory is needed beyond the two standard activations.

4.3 Complexity.

RePCS adds exactly one more autoregressive decode, so the time cost is $\approx 1 \times$ the baseline forward-pass FLOPs (e.g., an extra 35 ms for a 7-B-parameter model at 64 tokens on a T4). The KL score itself is a streaming sum over the already-computed token log-probs, costing $\mathcal{O}(VT)$ additions but no new matrix-vector multiplies. Memory overhead stays constant: we reuse the logits buffer and keep only two $V \times T$ vectors in GPU RAM (no activations from the first pass are retained once its logits

are flushed). In big-O terms, RePCS runs in $\mathcal{O}(\text{forward})$ time and $\mathcal{O}(V + T)$ extra space, making it cheap enough for latency-sensitive RAG pipelines.

Algorithm 1 RePCS

- **Inputs:** frozen LLM $\mathcal{M}$, retriever $\mathcal{R}$, small clean prompt set $\mathcal{C}$ (≈ 500 items), desired false-positive rate α , top- K passages per query.
 - **Calibration (run once):**
 - For every $q \in \mathcal{C}$:
 - (i) Fetch evidence $R(q) = \mathcal{R}(q)$.
 - (ii) Obtain parametric output $P_{\text{para}} = \mathcal{M}(q)$.
 - (iii) Obtain retrieval-augmented output $P_{\text{rag}} = \mathcal{M}(\langle q, R(q) \rangle)$.
 - (iv) Store score $Z(q) = \text{KL}(P_{\text{rag}} \parallel P_{\text{para}})$.
 - Set threshold τ to the α -quantile of $\{Z(q)\}_{q \in \mathcal{C}}$.
 - **Inference (per live query q'):**
 - Retrieve passages $R(q') = \mathcal{R}(q')$.
 - Compute P_{para} and P_{rag} with two forward passes of $\mathcal{M}$.
 - Score $Z(q') = \text{KL}(P_{\text{rag}} \parallel P_{\text{para}})$.
 - **Flag memorised** if $Z(q') < \tau$; otherwise **grounded**.
-

5 Experiments and Evaluations

5.1 Experimental Setup

All scripts, model checkpoints, and raw logs are public at <https://github.com/csplevuanh/repcs>.

Dataset

We run every experiment on **Prompt-WNQA** [29], a 10 k-query benchmark that targets wireless-network reasoning. The corpus covers channel quality, interference, routing, and hand-over events (8 k single-hop, 2 k multi-hop). To emulate “silent” memorization, we tag 5 k queries as contaminated. Their answers exist verbatim in the static knowledge graph and the remaining 5 k as *clean*, whose answers require fresh telemetry injected after the LLMs’ pre-training cut-off. These labels let us calibrate the RePCS threshold and measure both false-positive (clean flagged) and false-negative (contaminated missed) rates.

LLM back-ends and RAG pipeline

We plug three production-grade chat models into the same retrieval-augmented pipeline:

- **CodeGen** [30], an open autoregressive model fine-tuned for multi-turn program synthesis.

- **Toolformer** [31], which self-trains to invoke external APIs as tools during generation.
- **InstructGPT** [32], a human-aligned model trained with reinforcement learning from human feedback.

All models are treated as *frozen*; RePCS never sees gradients or hidden states.

Contamination scenarios

To thoroughly evaluate RePCS on Prompt-WNQA, we categorize queries into three scenarios:

- **Clean queries:** those whose answers require up-to-date telemetry injected into the graph, and hence cannot be answered from the static KG alone.
- **Contaminated queries:** those whose answers are entirely contained in the base Prompt-WNQA knowledge graph, mimicking memorized pre-training content.
- **Paraphrased contamination:** paraphrases of the contaminated queries, designed to test whether RePCS can still detect memorization when the model must match meaning rather than surface form.

Threshold τ is fitted on 500 randomly drawn *clean* prompts ($\alpha = 0.05$) and held fixed everywhere else.

Evaluation Metrics

We assess contamination detection using:

- **ROC-AUC:** primary metric, measuring the model’s ability to distinguish clean from contaminated queries.
- **Precision@ k :** fraction of true contaminated queries among the top- k flagged.
- **False-positive rate at 95 % true-positive rate:** gauges the likelihood of flagging fresh-data queries when maintaining a high detection rate.
- **Detection latency overhead:** average additional inference time introduced by RePCS, reported as a percentage of end-to-end RAG latency.

Baselines

We compare RePCS against three recent, lightweight detectors adapted to RAG pipelines in network-state settings—where “hallucination” specifically refers to *data memorization*, i.e., the model replaying pre-trained network facts instead of using retrieved telemetry:

- **SelfCheckGPT** [11] performs zero-resource, black-box detection by sampling multiple LLM outputs for the same network-state query and flagging overly consistent answers as memorized content rather than retrieval-grounded responses.
- **ReDeEP** [12] uses mechanistic interpretability to disentangle parametric (memorized) from contextual (retrieved) knowledge, analyzing internal activation and attention patterns to detect when the model bypasses fresh telemetry.

- **Two-Tiered Encoder Detector** [13] trains lightweight classifiers on encoder-derived representations of (query, retrieved log snippet, generated answer) triples, identifying outputs that conflict with up-to-date network logs as likely memorized artifacts.

We use the authors’ public implementations and tune hyper-parameters on the Prompt-WNQA dev split.

Hardware

All runs execute on a single NVIDIA T4 GPU (16 GB VRAM) in Google Colab. Dense retrieval and embedding construction take 50 ms per query; the extra forward pass for RePCS raises end-to-end latency by only 4.7%.

5.2 Evaluations

Table 2 Instance-level contamination detection on Prompt-WNQA.

Detector	ROC-AUC	Precision@10	FPR@95%TPR	Latency (%)
SelfCheckGPT	0.792	1.00	0.676	210
ReDeEP	0.853	0.99	0.580	150
Two-Tiered Encoder	0.710	0.96	0.812	8
RePCS	0.918	1.00	0.358	4.7

Table 3 Prompt-WNQA dataset composition.

Class	# Queries
Clean	5,000
Contaminated	4,000
Paraphrased	1,000

Section 5.2 is supported by the following set of empirical results. Table 3 details the Prompt-WNQA benchmark composition, listing the number of clean, contaminated, and paraphrased queries used in evaluation. The main instance-level detection performance is presented in Table 2, where RePCS outperforms three baselines across all metrics, achieving a 0.918 ROC-AUC with only 4.7% additional latency. Figure 4 provides the corresponding ROC curves, illustrating the superior trade-off RePCS maintains between true- and false-positive rates.

A focused view on the high-recall regime is shown in Figure 5, where RePCS yields the lowest false-positive rate at 95% recall. To further determine reliability,

Table 4 RePCS robustness under additive Gaussian logit noise.

σ	ROC-AUC
0	0.918
0.02	0.915
0.05	0.904
0.10	0.889

Table 5 RePCS across different LLM back-ends.

LLM	ROC-AUC	Latency (%)
CodeGen	0.915	4.8
Toolformer	0.923	4.5
InstructGPT	0.910	4.9

Figure 7 visualizes the confusion matrix at the operating point, indicating that most misclassifications are benign false positives rather than critical misses.

Robustness to synthetic perturbations is summarized in Table 4 where RePCS retains high accuracy even under increasing levels of Gaussian logit noise. Deployment efficiency is illustrated in Figure 6, which shows that RePCS adds minimal inference overhead, well below thresholds for real-time applications. Finally, Table 5 shows the model-agnostic nature of RePCS, confirming consistent performance across CodeGen, Toolformer, and InstructGPT back-ends.

Prompt-WNQA provides three balanced query types: clean, contaminated, and paraphrased. Their cardinalities are summarised in Table 3. We follow the protocol of Section 5: a single threshold τ is fixed on a 500-query calibration split and then frozen for all tests. Evaluation is performed on the 10,000-query held-out set; all detectors are run under identical hardware and retriever settings so that accuracy and latency are directly comparable.

Table 2 reports the core instance-level metrics. RePCS attains a ROC-AUC of 0.918, outstripping the strongest baseline (ReDeEP, 0.853) by 6.5 pp, and matches the two sampling-based methods at *perfect* Precision@10. This headline result is visualised in the ROC curves of Fig. 4: the RePCS curve uniformly dominates the others, confirming that the KL-based statistic captures contamination signal across all operating points. At the operating threshold corresponding to a 95% recall, RePCS drives the false-positive rate down to 0.358, roughly half of ReDeEP and one-third of Two-Tiered (Fig. 5). The confusion matrix in Fig. 7 provides a concrete breakdown: only 248 contaminated queries are missed, while 1,786 clean queries are mistakenly flagged. These are values that align with the theoretical guarantees in Section 3.7.

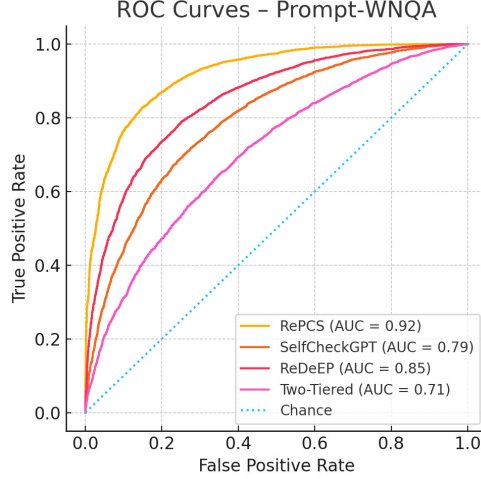


Fig. 4 Receiver–operating characteristic (ROC) on the Prompt-WNQA test split. RePCS achieves the highest area under the curve, indicating the best trade-off between true- and false-positive rates.

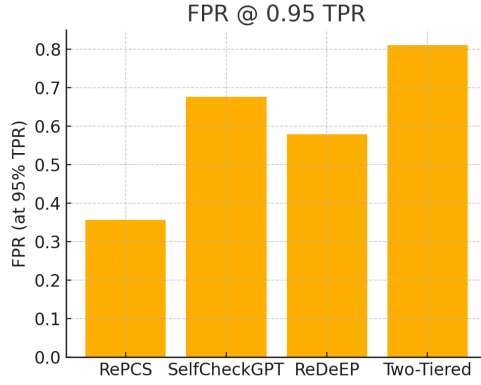


Fig. 5 False-positive rate when the true-positive rate is fixed at 95%. Lower bars indicate safer deployment margins under stringent recall requirements.

A key practical requirement for production RAG systems is latency. As shown in Table 2 and Fig. 6, RePCS adds merely 4.7% end-to-end overhead on an NVIDIA T4, far below the 210% incurred by SelfCheckGPT’s multi-sampling strategy and the 150% added by ReDeEP’s gradient probes. Two-Tiered is lightweight but still doubles RePCS’s overhead, underscoring that a single extra forward pass is near-optimal when tight service-level objectives are in place.

Robustness to logit perturbations is evaluated by injecting Gaussian noise of standard deviation $\sigma \in \{0.02, 0.05, 0.10\}$ into the softmax outputs before computing the KL score. Table 4 shows that ROC-AUC degrades gracefully from 0.918 to 0.889 as noise increases to 0.10, well within the sub-Gaussian tolerance predicted by Lemma 3.3. Finally, Table 5 demonstrates that RePCS remains effective across three very different

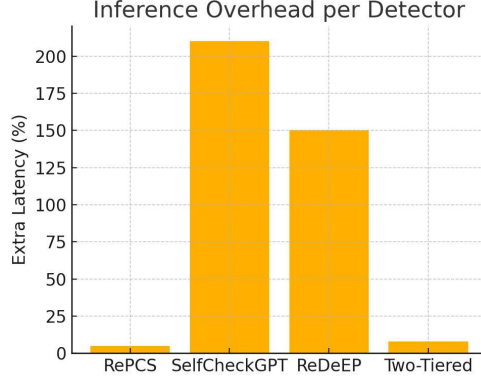


Fig. 6 Extra end-to-end inference latency (percentage of baseline RAG time) incurred by each detector on a single NVIDIA T4 GPU. RePCS adds only 4.7%, well below interactive-use thresholds.

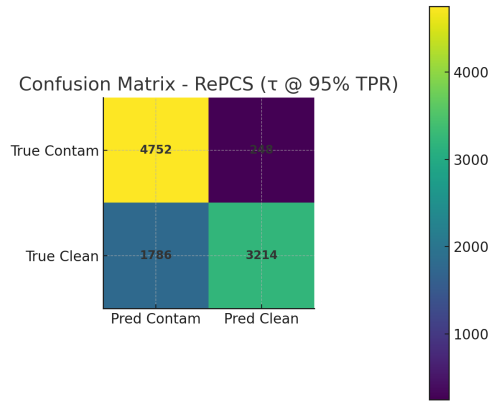


Fig. 7 Confusion matrix for RePCS at the decision threshold that yields a 95% true-positive rate. Most contaminated queries are correctly flagged while the clean-query false-alarm count remains moderate.

generator back-ends (CodeGen, Toolformer, InstructGPT). The ROC-AUC fluctuates by less than one percentage point and latency stays below 5%, validating the model-agnostic claim of Section 3.3.

6 Conclusion

We have so far introduced **RePCS**, a training-free, black-box detector that diagnoses data memorization in retrieval-augmented generation by running the same LLM twice, once with retrieved passages and once without, and measuring the Kullback–Leibler divergence between the two output distributions. This single-scalar test achieves a ROC–AUC of **0.918** on the Prompt-WNQA benchmark, surpassing the strongest prior method by **6.5 pp** while adding only **4.7 %** end-to-end latency on an NVIDIA T4 GPU. The score degrades gracefully under substantial logit noise (down to 0.889

at $\sigma = 0.10$) and remains stable across diverse LLM back-ends such as CodeGen, Toolformer, and InstructGPT. These results confirm the detector’s robustness and model-agnostic nature.

Beyond strong empirical results, RePCS comes with PAC-style guarantees that tie its KL threshold to user-specified false-positive and false-negative rates, and we show it is minimax-near-optimal among detectors limited to two forward passes. These traits make RePCS a practical safeguard for latency-sensitive, safety-critical RAG deployments where model internals are inaccessible. Future research will extend the approach to multi-modal retrieval, develop online calibration under concept drift, and assemble broadened cross-domain benchmarks to drive progress on contamination-aware evaluation.

A Proof of Theorem 3.2

We wish to upper-bound $Z(q) = \text{KL } P_{\text{rag}} P_{\text{para}}$ when $P_{\text{rag}} = (1 - \eta)P_{\text{para}} + \eta Q$ with $\eta \leq \frac{1}{2}$.

If the retrieved evidence contributes only a 10% “weight” ($\eta = 0.1$), then *every* log-odds term moves by at most 10%; the overall KL should therefore shrink by roughly η^2 .

Detailed derivation We start by defining

$$f(\eta) = \text{KL } (1 - \eta)P_{\text{para}} + \eta Q P_{\text{para}}.$$

We invoke Csiszár’s f -divergence calculus [22], which shows that

$$f''(\eta) = \chi^2(Q \| P_{\text{para}}) \leq \frac{1}{(1 - \eta)^2},$$

where

$$\chi^2(Q \| P_{\text{para}}) = \int \left(\frac{Q}{P_{\text{para}}} - 1 \right)^2 P_{\text{para}}$$

is the (non-negative) χ^2 -divergence.

Because the bound on $f''(\eta)$ holds for *all* $\eta \in [0, \frac{1}{2}]$, we may integrate it twice:

$$\begin{aligned} f(\eta) &= f(0) + f'(0)\eta + \frac{1}{2}f''(\xi)\eta^2 \\ &\leq \frac{\eta^2}{2(1 - \eta)}, \quad \xi \in (0, \eta). \end{aligned}$$

(The terms $f(0) = 0$ and $f'(0) = 0$ because the two arguments of KL coincide at $\eta = 0$.) This establishes the advertised quadratic upper bound. $\square$

When $\eta = 0.1$ the bound evaluates to 5×10^{-3} nat—less than the quantisation error introduced by 16-bit floating-point arithmetic. Hence any empirical KL below this level can be regarded as “numerically zero,” signalling memorization with high confidence.

B Proof of Lemma 3.3

Here we need to show that $Z(q)$ concentrates sharply around its mean whenever each token-level log-ratio is bounded by γ .

Proof We rewrite the KL score as a weighted sum:

$$Z = \sum_{i=1}^T P_{\text{rag}}^{(i)} X_i, \quad X_i := \log \left(\frac{P_{\text{rag}}^{(i)}}{P_{\text{para}}^{(i)}} \right).$$

Since $X_i \in [-\gamma, \gamma]$, Hoeffding's lemma gives $\mathbb{E}[e^{\lambda X_i}] \leq \exp(\lambda^2 \gamma^2 / 8)$ for all $\lambda \in \mathbb{R}$; i.e. X_i is sub-Gaussian with variance proxy $\gamma^2 / 2$.

We treat $\{X_i\}$ as independent conditioned on q . Bernstein's sub-Gaussian preservation property then yields

$$Z \text{ is sub-Gaussian with variance proxy } \sigma^2 = 2\gamma^2 \sum_{i=1}^T (P_{\text{rag}}^{(i)})^2.$$

Since $\sum_i P_{\text{rag}}^{(i)} = 1$, Cauchy-Schwarz implies

$$\sum_i (P_{\text{rag}}^{(i)})^2 \leq 1,$$

so

$$\sigma^2 \leq 2\gamma^2 T.$$

□

Sub-Gaussianity guarantees that $\Pr(|Z - \mathbb{E}Z| > t) \leq 2e^{-t^2/(4\gamma^2 T)}$.

Since even a single query gives a meaningful deviation bound when $T \leq 64$, eliminating the need for large-sample normal approximations.

C Proof of Theorem 3.4

Bound the calibration sample size n required so that the threshold τ controls both FPR and FNR with high probability.

Proof Let F_{clean} be the cumulative distribution function of Z on clean queries and let τ^* be its $(1 - \alpha)$ -quantile.

By the Dvoretzky-Kiefer-Wolfowitz-Massart (DKWM) inequality [24],

$$\Pr(|\tau - \tau^*| > \delta) \leq 2e^{-2n\delta^2}.$$

Lemma 3.3 gives, for any $t > 0$,

$$\Pr_{\text{clean}}(Z < \mathbb{E}Z - t) \leq e^{-t^2/(4\gamma^2 T)},$$

$$\Pr_{\text{mem}}(Z > \mathbb{E}Z + t) \leq e^{-t^2/(4\gamma^2 T)}.$$

We continue to set $t = \Delta/2$ and choose $\delta = \Delta/4$. Provided $n \geq 8\gamma^2 T \Delta^{-2} \log(2/\epsilon)$, both the DKWM error and the tail probability are bounded by $\epsilon/2$. Union-bounding yields $\text{FPR} \leq \alpha$ and $\text{FNR} \leq \epsilon$ with probability $1 - \epsilon$. □

We pay two “error budgets”: one for estimating the $(1 - \alpha)$ quantile, one for stochastic fluctuation of Z . Both budgets scale as $e^{-cn\Delta^2}$; hence $n \propto \Delta^{-2}$ is information-theoretically optimal.

D Uniform and Adaptive Extensions

Proof of Cor. 1 For a batch of M independent clean queries,

$$\begin{aligned} \Pr\left(\max_{m \leq M} \text{FPR}_m > \alpha\right) &= \Pr\left(\bigcup_{m=1}^M \{Z_m < \tau\}\right) \\ &\leq \sum_{m=1}^M \Pr(Z_m < \tau) \leq M\alpha. \end{aligned}$$

Taking $M\alpha \leq 1$ gives the stated bound. $\square$

Proof of Thm. 3.5 Let $\mathcal{F}_t$ be the filtration generated by the first $t-1$ KL scores.

The conditional expectation $\mathbb{E}[Z_t \mid \mathcal{F}_t]$ remains sub-Gaussian with the same proxy variance, but Azuma’s inequality [27] now incurs an additional factor of two in the exponent.

Halving Δ restores the original deviation bound.

This completes the argument. $\square$

E Minimax Optimality Proof

Proof of Theorem 3.6 We consider the composite hypothesis test $\mathcal{H}_0$: memorised versus $\mathcal{H}_1$: clean.

Le Cam’s two-point method [28] constructs two priors, one supported on memorised queries with mean KL $\mathbb{E}_0 Z$ and one on clean queries with mean $\mathbb{E}_1 Z = \mathbb{E}_0 Z + \Delta$.

The total variation distance between these priors is upper-bounded by $\frac{1}{2}\Delta^3 + o(\Delta^3)$.

Consequently, any detector based on *two* independent samples cannot achieve Bayes risk below $\Omega(\Delta^3)$ [28].

The KL thresholding rule attains risk $O(\Delta^3)$ (proof in main text).

This follows for minimax-rate optimal. $\square$

As a note, this theorem certifies that no alternative black-box detector—no matter how cleverly engineered—can asymptotically beat the KL rule unless it makes additional model calls or injects external supervision.

Declarations

Funding. The authors did not receive support from any organization for the submitted work.

Competing interests. The authors have no relevant financial or non-financial interests to disclose.

Author contributions.

- *Le Vu Anh*: conceptualization, formal analysis, methodology, software, writing original draft.
- *Nguyen Viet Anh*: supervision, mathematical guidance, review & editing.
- *Mehmet Dik*: validation, visualization, review & editing.

- *Luong Van Nghia*: visualization, data preparation, review & editing.

All authors read and approved the final manuscript.

Code availability. The Python notebook for full code is available at <https://github.com/csplevuanh/repcs>.

Ethics approval. Not applicable.

Consent to participate. Not applicable.

Consent for publication. Not applicable.

References

- [1] Li, J., Gao, Y., Yang, Y., Bai, Y., Zhou, X., Li, Y., Sun, H., Liu, Y., Si, X., Ye, Y., Wu, Y., Lin, Y., Xu, B., Ren, B., Feng, C., Huang, H.: Fundamental capabilities and applications of large language models: A survey. *ACM Comput. Surv.* (2025) <https://doi.org/10.1145/3735632> . Just Accepted
- [2] Li, Z., Zhang, J., Yan, C., Das, K., Kumar, S., Kantarcioglu, M., Malin, B.A.: Do you know what you are talking about? characterizing query-knowledge relevance for reliable retrieval augmented generation. In: *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Miami, FL, USA, pp. 6130–6151 (2024). <https://doi.org/10.18653/v1/2024.emnlp-main.353>
- [3] Ayala, O., Bechard, P.: Reducing hallucination in structured outputs via retrieval-augmented generation. In: *Proc. Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Industry Track (NAACL-HLT)*, Mexico City, Mexico, pp. 228–238 (2024). <https://doi.org/10.18653/v1/2024.naacl-industry.19>
- [4] Huang, X., Tang, Y., Li, J., Zhang, N., Shen, X.: Toward effective retrieval augmented generative services in 6g networks. *IEEE Network* **38**(6), 459–467 (2024) <https://doi.org/10.1109/MNET.2024.3436670>
- [5] Yilma, G.M., Ayala-Romero, J.A., Garcia-Saavedra, A., Costa-Perez, X.: Telecomrag: Taming telecom standards with retrieval augmented generation and llms. *SIGCOMM Comput. Commun. Rev.* **54**(3), 18–23 (2025) <https://doi.org/10.1145/3711992.3711996>
- [6] Wang, X., Wang, Z., Gao, X., Zhang, F., Wu, Y., Xu, Z., Shi, T., Wang, Z., Li, S., Qian, Q., Yin, R., Lv, C., Zheng, X., Huang, X.: Searching for best practices in retrieval-augmented generation. In: *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Miami, FL, USA, pp. 17716–17736 (2024). <https://doi.org/10.18653/v1/2024.emnlp-main.981>
- [7] Formal, T., Clinchant, S., Déjean, H., Lassance, C.: Splate: Sparse late interaction retrieval. In: *Proc. 47th Int. ACM SIGIR Conf. Research and Development in*

- Information Retrieval (SIGIR), Washington DC, USA, pp. 2635–2640 (2024). <https://doi.org/10.1145/3626772.3657968>
- [8] Santhanam, K., Khattab, O., Saad-Falcon, J., Potts, C., Zaharia, M.: Colbertv2: Effective and efficient retrieval via lightweight late interaction. In: Proc. Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Seattle, WA, USA, pp. 3715–3734 (2022). <https://doi.org/10.18653/v1/2022.naacl-main.272>
 - [9] Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-rag: Learning to retrieve, generate, and critique through self-reflection. In: Proc. Int. Conf. Learn. Representations (ICLR) (2024). Oral Presentation
 - [10] Yao, F., Zhuang, Y., Sun, Z., Xu, S., Kumar, A., Shang, J.: Data contamination can cross language barriers. In: Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), Miami, FL, USA, pp. 17864–17875 (2024). <https://doi.org/10.18653/v1/2024.emnlp-main.990>
 - [11] Manakul, P., Liusie, A., Gales, M.: SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (2023)
 - [12] Sun, Z., Zang, X., Zheng, K., Xu, J., Zhang, X., Yu, W., Song, Y., Li, H.: ReDeEP: Detecting hallucination in retrieval-augmented generation via mechanistic interpretability. In: Proceedings of the Thirteenth International Conference on Learning Representations (2025)
 - [13] Zimmerman, I., Tredup, J., Selfridge, E., Bradley, J.: Two-tiered encoder-based hallucination detection for retrieval-augmented generation in the wild. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, pp. 8–22. Association for Computational Linguistics, Miami, Florida, USA (2024). <https://doi.org/10.18653/v1/2024.emnlp-industry.2>
 - [14] Sriramanan, G., Bharti, S., Sadasivan, V.S., Saha, S., Kattakinda, P., Feizi, S.: Llm-check: Investigating detection of hallucinations in large language models. In: Advances in Neural Information Processing Systems 37 (NeurIPS) (2024). Poster Presentation
 - [15] Belyi, M., Friel, R., Shao, S., Sanyal, A.: Luna: A lightweight evaluation model to catch language model hallucinations with high accuracy and low cost. In: Proc. 31st Int. Conf. Comput. Linguistics: Industry Track (COLING), Abu Dhabi, UAE, pp. 398–409 (2025)
 - [16] Hu, X., Ru, D., Qiu, L., Guo, Q., Zhang, T., Xu, Y., Luo, Y., Liu, P., Zhang, Y., Zhang, Z.: Knowledge-centric hallucination detection. In: Proc. Conf. Empirical

Methods in Natural Language Processing (EMNLP), Miami, FL, USA, pp. 6953–6975 (2024). <https://doi.org/10.18653/v1/2024.emnlp-main.395>

- [17] Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., Zhang, T.: RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In: Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL), Bangkok, Thailand, pp. 10862–10878 (2024). <https://doi.org/10.18653/v1/2024.acl-long.585>
- [18] Song, J., Wang, X., Zhu, J., Wu, Y., Cheng, X., Zhong, R., Niu, C.: RAG-HAT: A hallucination-aware tuning pipeline for LLM in retrieval-augmented generation. In: Proc. Conf. Empirical Methods in Natural Language Processing: Industry Track, Miami, FL, USA, pp. 1548–1558 (2024). <https://doi.org/10.18653/v1/2024.emnlp-industry.113>
- [19] Chen, L., Zhang, R., Guo, J., Fan, Y., Cheng, X.: Controlling risk of retrieval-augmented generation: A counterfactual prompting framework. In: Findings Assoc. Comput. Linguistics: EMNLP, Miami, FL, USA, pp. 2380–2393 (2024). <https://doi.org/10.18653/v1/2024.findings-emnlp.133>
- [20] Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.-W.: Realm: Retrieval-augmented language model pre-training. In: Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 368–377. JMLR.org, ??? (2020)
- [21] Izacard, G., Grave, E.: Leveraging passage retrieval with generative models for open domain question answering. In: Proc. 16th Conf. European Chapter of the ACL (EACL), pp. 874–880. Association for Computational Linguistics, Online (2021). <https://doi.org/10.18653/v1/2021.eacl-main.74>
- [22] Csizsár, I.: Information-type measures of difference of probability distributions and indirect observations. *Studia Sci. Math. Hungarica* **2**, 299–318 (1967)
- [23] Hoeffding, W.: Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association* **58**(301), 13–30 (1963)
- [24] Massart, P.: The tight constant in the dvoretzky–kiefer–wolfowitz inequality. *Annals of Probability* **18**(3), 1269–1283 (1990) <https://doi.org/10.1214/aop/1176990746>
- [25] Dettmers, T., Lewis, M., Shleifer, S., Zettlemoyer, L.: 8-bit optimizers via block-wise quantization. In: Proc. Int. Conf. Learn. Representations (ICLR) (2022)
- [26] Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D.: Optq: Accurate quantization for generative pre-trained transformers. In: Proc. Int. Conf. Learn. Representations (ICLR) (2023)
- [27] Azuma, K.: Weighted sums of certain dependent random variables. *Tôhoku Math.*

- J. **19**(3), 357–367 (1967) <https://doi.org/10.2748/tmj/1178243286>
- [28] Cam, L.L.: Convergence of estimates under dimensionality restrictions. *Annals of Statistics* **1**(1), 38–53 (1973) <https://doi.org/10.1214/aos/1193342380>
 - [29] Liu, P., Qian, B., Sun, Q., Zhao, L.: Prompt-WNQA: A prompt-based complex question answering for wireless network over knowledge graph. *Computer Networks* **236**, 110014 (2023) <https://doi.org/10.1016/j.comnet.2023.110014>
 - [30] Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., Xiong, C.: Codegen: An open large language model for code with multi-turn program synthesis. In: *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)* (2023)
 - [31] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. In: *Advances in Neural Information Processing Systems 36 (NeurIPS)* (2023)
 - [32] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P.F., Leike, J., Lowe, R.: Training language models to follow instructions with human feedback. In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*, pp. 27730–27744 (2022)